

Experiment 3: Installation and Configuration of DVC, Dataset Versioning, and Integration with Git

Aim

To install and configure Data Version Control (DVC), track and version datasets, manage storage of data artifacts, compare different dataset versions, integrate DVC with Git for a unified code-and-data versioning workflow, and demonstrate the reproducibility of Machine Learning experiments using versioned data.

Prerequisites & Tool Stack

- Completed Experiment 2 (Git-initialized `mlops-iris-classifier` repository)
- Python 3.9+ and `pip`
- DVC (`dvc`, version 3.x)
- A DVC-supported remote storage backend (local folder remote used here for simplicity; cloud options: Amazon S3, Google Cloud Storage, Azure Blob)
- Git (already installed in Experiment 2)

Theoretical Background

Data Version Control (DVC) is an open-source tool purpose-built to solve the problem Git cannot: efficiently versioning large data files and machine learning models alongside code. As discussed in Experiment 2, Git's internal object model is optimized for small, text-based diffs; storing a 2 GB dataset directly in a Git repository would bloat the `.git` directory on every change and make cloning painfully slow. DVC solves this by separating **metadata** from **content**. When a file or directory is added to DVC (`dvc add`), DVC computes a content hash (MD5) of the data, moves the actual data into a local **cache** (`.dvc/cache`), and creates a small, human-readable `.dvc` metafile (YAML) containing the hash, size, and path. This tiny `.dvc` file — not the data itself — is what gets committed to Git. This design means Git tracks the *history of which data version was in use at each code commit*, while the (potentially huge) data itself lives outside Git, in a **DVC remote**.

A **DVC remote** is any storage backend (a shared network drive, S3 bucket, GCS bucket, SSH server, or even a second local directory) configured to hold the actual content that `.dvc` files point to. Running `dvc push` uploads content from the local cache to the configured remote; `dvc pull` downloads it back down using the hash pointer in the `.dvc` file — conceptually mirroring `git push/git pull`, but for data. This means a teammate can `git clone` the repository (getting all code and lightweight `.dvc` pointer files instantly) and then `dvc pull` to fetch only the specific data versions referenced by the commit they checked out.

Dataset version comparison and switching follows directly from this design: since every historical Git commit contains a `.dvc` file pointing to a specific data hash, checking out an old Git commit (`git checkout <commit>`) automatically changes which `.dvc` pointer is present; running `dvc checkout` afterward then syncs the actual working-directory data to match that historical hash — instantly reverting your dataset to exactly what it was at that point in project history, without keeping duplicate full copies in your working directory beyond what DVC's cache (via reflinks/hardlinks) requires.

Integration with Git is therefore not optional glue but DVC's core design principle: DVC is built to be used *alongside* Git, not as a replacement. Every meaningful data change is expected to be followed by `dvc add + git add *.dvc + git commit`, producing a Git history where each commit unambiguously identifies both the exact code and the exact dataset that were used together — this pairing is what makes an experiment fully **reproducible**: given any commit hash, a collaborator (or a CI pipeline) can `git checkout + dvc pull` and be certain of retrieving the exact code-data pair that produced a historical result, rather than approximate or “best guess” reconstructions. DVC additionally supports **DVC Pipelines** (`dvc.yaml`), which formally encode data-processing/training stages with declared dependencies and outputs, allowing DVC to detect exactly which stages need re-execution when an upstream dependency (code or data) changes — this is explored further in Experiment 4.

Step-by-Step Implementation Procedure

Step 1: Install DVC

```
pip install dvc
```

`dvc --version` checks whether DVC is installed and displays the installed DVC version.

if you get error `dvc command not found` then

try running `/c/Users/Admin/AppData/Local/Python/pythoncore-3.14-64/Scripts/dvc.exe -version`

if you get 3.67.1

then in environment variables, under variables for admin, select path, click edit, click new, `C:\Users\Admin\AppData\Local\Python\pythoncore-3.14-64\Scripts` click ok.

Restart git bash and then run `dvc --version`

Expected output resembles:

```
3.51.2
```

Step 2: Initialize DVC in the Existing Git Repository

```
cd mlops-iris-classifier
```

`cd` means **Change Directory**. It moves the terminal into the project folder where Git and DVC will be used.

```
dvc init
```

initializes DVC inside an existing Git repository. It creates the .dvc directory and DVC configuration files needed for data versioning.

```
git status
```

git status shows:

- the current branch
- files that are staged
- modified files
- untracked files

dvc init creates a .dvc/ directory (containing .dvc/config, .dvc/cache/, and .dvc/.gitignore) and stages these for Git automatically:

Stage DVC Files

```
git add .dvc .dvcignore
```

git add stages files so that they can be included in the next Git commit.

Here, the DVC configuration files are being added to Git.

Commit DVC initialization

```
git commit -m "chore: initialize DVC in the repository"
```

git commit permanently records the staged changes in Git history.

The -m option provides the commit message

Step 3: Configure a DVC Remote (Local Folder Remote)

For classroom use, a local folder simulates a shared/cloud remote:

A DVC remote is a location where DVC stores the actual data objects.

For this experiment, a local folder is used instead of cloud storage.

```
mkdir -p ~/dvc-remote-storage
```

`mkdir` = make directory

`-p` = create the directory if it does not already exist

`~/dvc-remote-storage` = the remote storage directory inside your user's home directory

```
dvc remote add -d myremote ~/dvc-remote-storage
```

This configures a DVC remote named `myremote`.

The `-d` option means: Make this the **default DVC remote**.

stage remote configuration

```
git add .dvc/config
```

It stages the modified `.dvc/config` file so that the remote configuration can be stored in Git.

Commit remote configuration

```
git commit -m "chore: configure local DVC remote"
```

It records the DVC remote configuration in Git history.

(For a real cloud remote, e.g. S3: `dvc remote add -d myremote s3://my-bucket/dvc-store` followed by AWS credential configuration.)

Step 4: Create and Track a Sample Dataset

Generate a raw dataset file to version:

```
mkdir -p data/raw
```

 This directory will contain the raw dataset.
`code src/generate_data.py` code opens the specified file in Visual Studio Code.

```
python src/generate_data.py
```

It executes the Python script that generates the Iris dataset and saves it as: `data/raw/iris_v1.csv`

add this code to `generate_data.py`

```
# scripts/generate_data.py
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame
```

```
df.to_csv("data/raw/iris_v1.csv", index=False)
print(f"Saved {len(df)} rows to data/raw/iris_v1.csv")
```

`ls data/raw` lists the files and directories present in the specified location

Track the dataset with DVC:

```
dvc add data/raw/iris_v1.csv
```

`dvc add` tells DVC: "Start tracking this dataset."

DVC calculates a content hash for the dataset and creates a small `.dvc` metafile. It does not put the entire CSV into Git.

DVC therefore created: `data/raw/iris_v1.csv.dvc` and a `.gitignore` file under `data/raw`.

This produces `data/raw/iris_v1.csv.dvc`, e.g.:

```
outs:
- md5: 9f8b1e2c3a4d5e6f7a8b9c0d1e2f3a4b
  size: 3867
  hash: md5
  path: iris_v1.csv
```

Commit the metafile (not the data) to Git:

```
git add data/raw/iris_v1.csv.dvc data/raw/.gitignore
```

It stages the DVC tracking information, not the actual CSV dataset.

This is the key Git-DVC relationship:

Actual dataset

↓

DVC tracks it

`.dvc` metafile

↓

Git tracks it

Commit dataset version 1

```
git commit -m "data: add iris_v1 raw dataset (150 rows) tracked via DVC"
```

It stores the first dataset version's DVC metadata in Git.

Push dataset to DVC Remote

```
dvc push
```

uploads the DVC-tracked data from the local DVC cache to the configured DVC remote.

Working Dataset

↓

DVC Cache

↓

```
dvc push
```

↓

DVC Remote

Git does **not** receive the CSV itself.

Step 5: Modify the Dataset and Create a New Version

Simulate a data update (e.g., an upstream data source adds 20 synthetic augmented rows):

The dataset is modified from:

150 rows → 170 rows

using 20 synthetic augmented rows.

code src/augment_data.py

add this code to augment_data.py

```
# src/augment_data.py
import pandas as pd
import numpy as np

df = pd.read_csv("data/raw/iris_v1.csv")
rng = np.random.default_rng(42)
noise = rng.normal(0, 0.05, size=(20, 4))
sample = df.sample(20, random_state=42).reset_index(drop=True)
```

```
sample.iloc[:, :4] = sample.iloc[:, :4].values + noise
augmented = pd.concat([df, sample], ignore_index=True)
augmented.to_csv("data/raw/iris_v1.csv", index=False)
print(f"Dataset now has {len(augmented)} rows")
```

```
python src/augment_data.py
```

It reads the existing 150-row dataset, creates 20 synthetic rows, adds them to the dataset, and overwrites: data/raw/iris_v1.csv

```
wc -l data/raw/iris_v1.csv
```

 count the number of lines in specified file

[171 data/raw/iris_v1.csv -----output of wc -l data/raw/iris_v1.csv

150 original rows

+ 20 new rows

170 data rows

+ 1 header

----- 171 lines]

Track Modified dataset with DVC

```
dvc add data/raw/iris_v1.csv
```

DVC detects that the contents of the CSV have changed.

It calculates a **new content hash** and updates: data/raw/iris_v1.csv.dvc

Stage the updated DVC metafile

```
git add data/raw/iris_v1.csv.dvc
```

It stages the updated .dvc file containing the new dataset hash.

Again, Git is tracking the pointer/metadata, not the 170-row CSV itself.

Commit Dataset Version 2

```
git commit -m "data: augment iris dataset with 20 synthetic rows (150  
-> 170)"
```

It creates a second Git commit containing the updated DVC metadata.

Push version 2 to DVC Remote

```
dvc push
```

It uploads the new DVC data object to the remote storage.

The first dataset version remains in the remote, while the new dataset object is added.

Step 6: Compare Dataset Versions

```
git log --oneline -- data/raw/iris_v1.csv.dvc
```

It shows the Git commits that changed the DVC metafile for iris_v1.csv.

The options mean:

- `git log` → show commit history
- `--oneline` → show each commit in a compact one-line format
- `-- data/raw/iris_v1.csv.dvc` → show history only for this file

Expected output:

```
6b8d972 (HEAD -> main) data: augment iris  
dataset with 20 synthetic rows (150 -> 170)
```

```
602aa4a data: add iris_v1 raw dataset (150 rows) tracked via DVC
```

Therefore, we have exactly **two dataset versions**:

Version	Git commit	Dataset
---------	------------	---------

Version 1	602aa4a	150 rows
-----------	---------	----------

Version 2	6b8d972	170 rows
-----------	---------	----------

Use `dvc diff` to compare the current working data against a previous commit:

```
dvc diff 602aa4a
```


`dvc diff` compares the DVC-tracked data between a specified Git commit and the current workspace.

Here:

602aa4a

↓

Version 1

↓

150 rows

Current HEAD

↓

Version 2

↓

170 rows

Expected output:

Modified:

data/raw/iris_v1.csv

files summary: 1 modified

Compare the DVC Hashes

To view the two `.dvc` metafile contents side by side and confirm the underlying hash changed:

View Version 1's DVC metafile

```
git show 602aa4a:data/raw/iris_v1.csv.dvc
```

`git show` displays the contents of a file from a particular Git commit.

Here we are asking:

"Show me the DVC metafile for `iris_v1.csv` as it existed in commit `602aa4a`."

outs:

```
- md5: 21d441a28bce4417276097df955afc50 version 1 Hash
  size: 2928 version 1 size
  hash: md5
  path: iris_v1.csv
```

View Version 2's DVC metafile

```
git show HEAD:data/raw/iris_v1.csv.dvc
```

HEAD refers to the current/latest Git commit.

Therefore, this command displays the DVC metafile for the latest dataset version.

outs:

```
- md5: 674c8c36bb7c4ba8d851dee9e6ee67af version 2 Hash
  size: 4462 version 2 size
  hash: md5
  path: iris_v1.csv
```

Step 7: Reproduce a Historical Experiment (Data + Code Together)

The goal is to temporarily restore **Version 1**.

Get Version 1's .dvc pointer

```
# Revert working directory to the exact code + data state of the first commit
git checkout 602aa4a -- data/raw/iris_v1.csv.dvc
```

It takes only the .dvc metafile from commit 602aa4a and places that older version into the working directory.

It does not yet restore the actual CSV.

The .dvc file now points to the Version 1 hash.

Restore the actual dataset using DVC

```
dvc checkout data/raw/iris_v1.csv.dvc
```

dvc checkout reads the hash from the .dvc file and retrieves the corresponding data object from the DVC cache.

In simple terms:

Old .dvc file

↓

Old MD5 hash

↓

DVC cache

↓

Old dataset restored

Verify Version 1

```
wc -l data/raw/iris_v1.csv    # confirms 151 lines (150 rows + header)
restored
```

Restore the Latest Dataset Version

Get the latest .dvc pointer

Return to the latest state

```
git checkout HEAD -- data/raw/iris_v1.csv.dvc
```

It replaces the old Version 1 .dvc file with the latest .dvc file from HEAD.

Therefore, the pointer once again refers to Version 2.

Restore Version 2

```
dvc checkout data/raw/iris_v1.csv.dvc
```

It reads the latest hash from the .dvc file and restores the corresponding dataset.

Verify Version 2

```
wc -l data/raw/iris_v1.csv    # confirms 171 lines (170 rows + header)
restored
```

Step 8: Document the DVC Workflow

Create docs/DVC_WORKFLOW.md summarizing: remote configuration used, the dvc add → git add → git commit → dvc push cycle followed for every data change, and how dvc diff / dvc checkout were used to compare and restore dataset versions.

docs/DVC_WORKFLOW.md

`git add docs/DVC_WORKFLOW.md` It stages the DVC workflow documentation so it can be committed to Git.

`git commit -m "docs: add DVC workflow documentation"` It saves the DVC workflow documentation in Git history.

`git push origin main` It uploads your local Git commits to the remote GitHub repository.

`git add src/augment_data.py src/generate_data.py`

`git commit -m "feat: add dataset generation and augmentation scripts"`

`git push origin main`

`git status`

Additional Verification – Current DVC Metafile

`cat data/raw/iris_v1.csv.dvc` cat displays the contents of a file directly in the terminal.

Additional Verification – Dataset Version History

`git log --oneline -- data/raw/iris_v1.csv.dvc` This confirms that the DVC metafile has exactly two dataset-version commits.

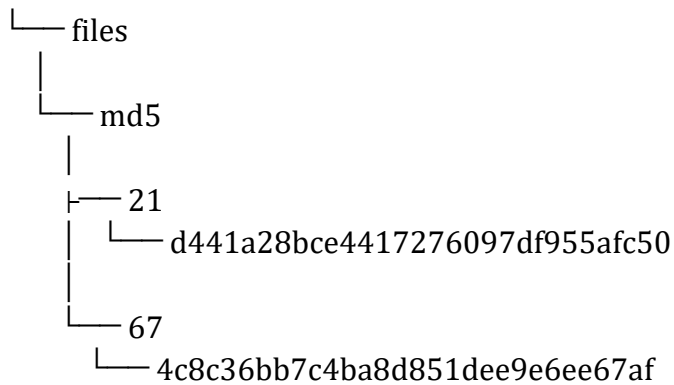
Verification & Execution Steps

- `dvc --version` returns a valid 3.x version string.
- `ls .dvc/cache` (or `dvc cache dir`) shows populated cache directories containing hashed data objects after `dvc add`.
- `git log --oneline -- data/raw/iris_v1.csv.dvc` shows exactly 2 commits, confirming two tracked dataset versions.
- `dvc diff a1b2c3d` (using your actual first commit hash) reports `data/raw/iris_v1.csv` as Modified.
- `ls ~/dvc-remote-storage` (the local remote) contains hashed object subdirectories after `dvc push`, confirming data was uploaded to the remote independently of Git.
- After `dvc checkout` to the older `.dvc` pointer, `wc -l data/raw/iris_v1.csv` reports 151 lines; after switching back, it reports 171 lines — confirming DVC correctly restores each historical data version on demand.
- `docs/DVC_WORKFLOW.md` exists and accurately documents the remote and workflow used.

DVC Remote Storage structure

C:\Users\Admin\dvc-remote-storage

|



What are those two files?

Those long names are **MD5 content hashes**.

They are not random filenames. DVC essentially does:

iris_v1.csv



calculate MD5 hash



21d441a28bce4417276097df955afc50



store it as:

files/md5/21/d441a28bce4417276097df955afc50

The first two characters become the folder name:

21 / d441a28bce4417276097df955afc50

Similarly:

67 / 4c8c36bb7c4ba8d851dee9e6ee67af

And the really interesting part...

You have **two hashes**, which strongly indicates that DVC has stored **two different versions/objects**.

Most likely:

Version 1

150 rows



hash → 21d441a28bce4417276097df955afc50

Version 2

170 rows



hash → 674c8c36bb7c4ba8d851dee9e6ee67af

DVC Remote Storage — What Your Two Hashes Mean

Your experiment produced two different DVC objects.

Version 1

MD5:

21d441a28bce4417276097df955afc50

Size:

2928 bytes

Version 2

MD5:

674c8c36bb7c4ba8d851dee9e6ee67af

Size:

4462 bytes

These values are recorded in the `.dvc` metafile for each version.

The important concept is:

iris_v1.csv

|

▼

DVC calculates content hash

|

|— Version 1

|

↓

| 21d441a28bce4417276097df955afc50

|

|— Version 2

↓

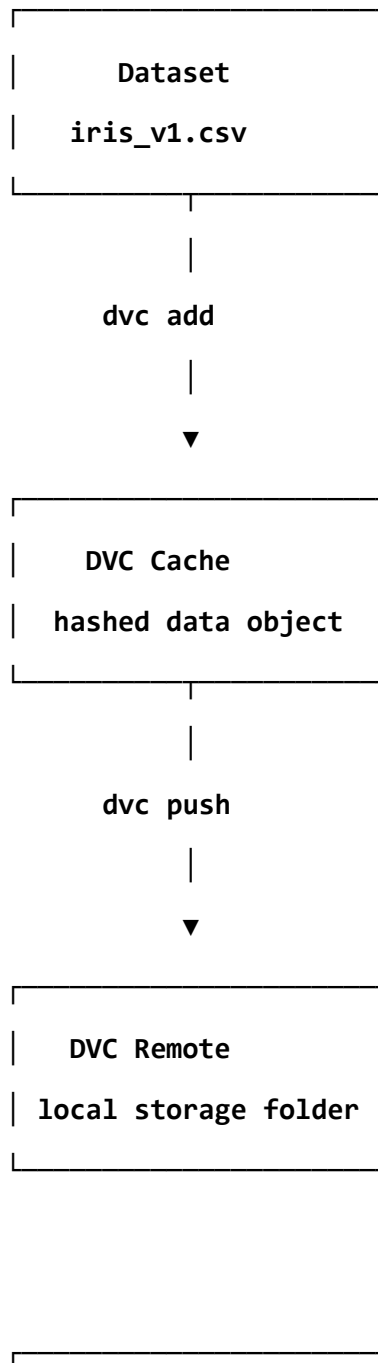
674c8c36bb7c4ba8d851dee9e6ee67af

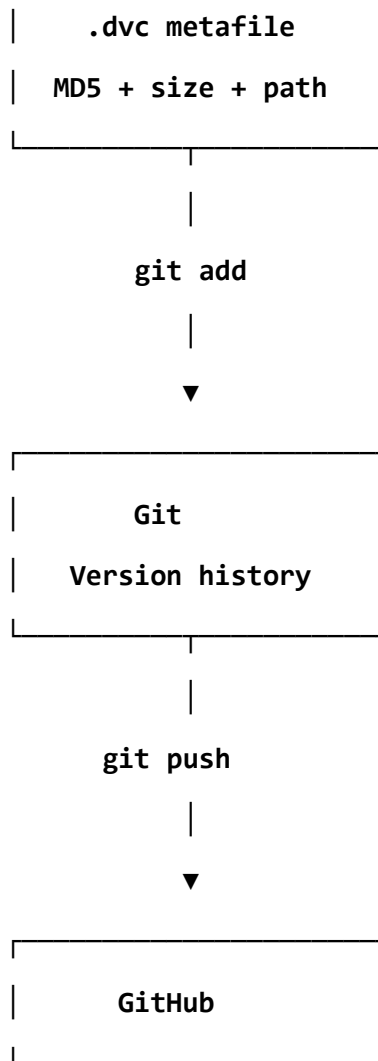
Because the dataset contents changed, the hash changed.

So DVC can distinguish the two dataset versions even though the filename remained iris_v1.csv.

Complete Git + DVC Workflow Demonstrated

Your actual experiment demonstrated this workflow:





Important Commands and Their One-Line Purpose

Command	Purpose
<code>dvc --version</code>	Displays installed DVC version
<code>dvc init</code>	Initializes DVC in a Git repository
<code>git status</code>	Shows repository/staging status
<code>git add</code>	Stages files for a Git commit

Command	Purpose
git commit	Saves staged changes to Git history
mkdir -p	Creates directories
dvc remote add -d	Configures the default DVC remote
code file.py	Opens a file in VS Code
python file.py	Executes a Python script
ls	Lists files/directories
dvc add	Starts tracking a dataset with DVC
dvc push	Uploads DVC data objects to remote storage
wc -l	Counts lines in a file
git log --oneline	Displays compact Git commit history
dvc diff	Compares DVC-tracked data between versions
git show	Displays a file/content from a particular Git commit
git checkout <commit> -- <file>	Restores a specific file from a Git commit
dvc checkout	Restores the actual data corresponding to a .dvc pointer
git push origin main	Uploads Git commits to GitHub
cat file	Displays file contents in the terminal

Viva-Voce / Post-Lab Questions

Q1. Explain how DVC achieves both the efficient storage of large files and full compatibility with Git's commit history, using the concept of "content-addressable storage." Answer: DVC uses content-addressable storage: every tracked file's content is hashed (MD5 by default), and the file is stored in the cache/remote under a path derived from that hash, rather than under its original filename. The lightweight `.dvc` metafile committed to Git contains only this hash, the file size, and its logical path — never the raw content. Because Git only ever sees small, fast-changing text metafiles, repository size and clone/checkout speed remain unaffected by dataset size, while Git's commit history still fully captures *which exact data version* (via its unique hash) corresponds to every commit, since a new/changed dataset produces a new hash and therefore a diff in the `.dvc` file that Git commits normally.

Q2. Why is it insufficient to store multiple named copies of a dataset directly in Git (e.g., `iris_v1.csv`, `iris_v2.csv`, `iris_v3.csv`) instead of using DVC? Answer: Storing multiple full copies directly in Git means every copy is stored in full inside Git's object database and remains there permanently in the repository's history (even if later deleted, since Git retains blobs referenced by any historical commit), causing repository size to grow linearly and permanently with every dataset revision — quickly making the repository unusably large to clone. It also loses the crucial *code-data correspondence*: nothing forces `iris_v2.csv` to be referenced by the specific commit that used it during training, so a collaborator has no reliable way to know which named file version was actually used to produce a specific historical model result. DVC solves both problems: identical content is stored once in the cache/remote regardless of how many `.dvc` files point to it (deduplication via hashing), and each Git commit's `.dvc` file unambiguously pins the exact data version used at that point in history.

Q3. In Step 7, why was it necessary to run both `git checkout ... -- <file>.dvc` and `dvc checkout`, rather than just one of the two commands? Answer: These two commands operate at different layers. `git checkout <commit> -- data/raw/iris_v1.csv.dvc` only restores the *metafile* (the small YAML pointer containing the historical hash) from that commit into the current working directory and Git index — it does not touch the actual CSV data file at all, since Git never tracked that data directly. `dvc checkout` then reads whatever hash is currently in the `.dvc` metafile and synchronizes the actual working-directory data file to match that hash, pulling it from the local cache (or remote, if not cached locally) if needed. Without the `git checkout` step, the `.dvc` file would still point to the newest hash, so `dvc checkout` would have nothing to change; without the `dvc checkout` step, the `.dvc` pointer would be updated but the actual `iris_v1.csv` file on disk would remain unchanged, leaving code and data out of sync.